

Правила разработки компонента веб-приложения

Если вы разрабатываете **публичный компонент** (компонент, который используется за пределами вашей команды), для удобного доступа его необходимо оформлять отдельно от общего прикладного кода функциональной области в специальном репозитории с отдельным модулем, чтобы потребители вашего компонента не подключали лишних зависимостей вашей функциональной области.

Репозиторий должен иметь имя "engine" и располагаться внутри функциональной группы git.

Например, для

- <https://git.sbis.ru/message>
- <https://git.sbis.ru/contragents>

будет

- <https://git.sbis.ru/message/engine>
- <https://git.sbis.ru/contragents/engine>

соответственно.

Если компонент сложно отнести к какой-то конкретной функциональной группе, то его можно опубликовать под собственным именем в группе [engine](#).

Разработка контроля и демо-примеров

Шаг 1. Напишите код на языке TS

См. [пример кода](#).

- [Стандарты разработки на TS](#)
- [Часто возникающие проблемы](#)

Обратите внимание на то, как правильно описывать интерфейсы, классы и опции, какие методы делать публичными, а какие приватными.

Шаг 2. Включите контрол в состав библиотеки

См. [пример](#).

- [Работа с TS библиотеками](#)

Шаг 3. Покройте код юнит-тестами

См. [пример](#).

- [Подробная статья про юнит-тесты](#)

Код должен быть написан так, чтобы его можно было легко покрыть тестами. Для этого код необходимо разбивать на методы, имеющие четко определенные аргументы и результат работы, которые можно проверить.

Покрываем тестами только логику работы, то есть все, кроме непосредственной работы с DOM или другими браузерными объектами.

Шаг 4. Реализуйте верстку и стилевое оформление контроля

См. [пример шаблона](#) и [пример библиотеки](#).

- [Стандарт разработки CSS и LESS](#)
- [Как организовывать CSS-пакеты в рамках библиотек](#)

Less-файлы контроля соберите в библиотеки.



Важно!

Названия less-файлов контролов начинаются с подчеркивания. Такое название говорит билдеру less о том, что для этих файлов не надо генерировать отдельный css.

Все стили должны прилетать только в составе css-библиотеки. При разработке стилей необходимо активно пользоваться файлом с основными классами. Не забудьте добавить файл в зависимости своего контрола.

Шаг 5. Добавьте разметку для системы помощи

Чтобы на элементы контрола можно было навешивать маршруты подсказок (пошаговые инструкции для пользователей), необходимо добавить атрибут **data-name** с уникальным в рамках страницы значением. Это позволит технологам привязывать подсказки к нужным элементам интерфейса.

Для того чтобы подсказки работали корректно и не терялись при переносе между окружениями, значения data-name должны соответствовать следующим правилам:

- Уникальность — значение data-name должно быть уникальным на странице (или в пределах сущности, например, карточки). Если на странице есть несколько похожих элементов (например, строки в таблице), каждому из них нужно задать своё уникальное значение. Для этого к общему базовому имени добавляют идентификатор элемента (например, data-name="table-row-**{id}**"). Это необходимо, чтобы подсказка всегда указывала на правильный элемент.
- Независимость от стенда — значение не должно зависеть от окружения (не включать имя хоста, номер версии и т.п.), чтобы маршруты подсказок корректно работали при переносе между разработкой, тестом и продуктивом.
- Использование data-qa — элементы, уже имеющие атрибут data-qa, также могут использоваться для подсказок, если их значение уникально. Однако для новых компонентов рекомендуется явно добавлять data-name, чтобы отделить разметку для тестирования от разметки для подсказок.

▼ Пример кода

- WML

```
1 <div data-name="demo-hint__inputContainer" class="controls-padding-2xs">
2   <Controls.baseInput:Label caption="Label"/>
3   <Controls.input:Text placeholder="Поле ввода"/>
4 </div>
```

- React/TSX (JSX)

```
<button data-name="demo-hint__saveButton">Сохранить</button>
```

Особенности для кнопок, ссылок и меню

В подсказках можно запрограммировать автоматический клик на элементе. Алгоритм выбора таргета:

- Если размеченный элемент содержит **пункт меню** – клик генерируется на нём.
- Если содержит **ссылку** или **кнопку**, и у них нет атрибута data-name с префиксом isolatedClick- – клик генерируется на ссылке/кнопке.
- В остальных случаях – клик генерируется на самом размеченном элементе.

Чтобы предотвратить автоматический клик на вложенной ссылке/кнопке, установите этому вложенному элементу атрибут data-name с префиксом isolatedClick- (например, data-name="isolatedClick-no").

Подробнее см. в документации [Система помощи. Маршруты подсказок](#).

Шаг 6. Добавьте темы

См. [пример](#).

- [Подробная статья про темы оформления](#)
- [Правила навешивания темизированных классов](#)

Все наши контролы, в зависимости от приложения, в котором они используются, могут иметь разные стиливые оформления. Такие виды оформления называются **темами**.

Новые переменные для контрола добавляются в [файл](#) и согласуются с [Бегуновым А.](#)

Шаг 7. Напишите автодокументацию и добавьте ее в библиотеку

См. [пример автодокументации](#) и [пример добавления в библиотеку](#).

- [Правила написания автодокументации](#)

Обратите внимание на:

- описание контрола;
- @class, @public;
- @implements — все интерфейсы должны быть описаны;
- все опции, которые добавляются в самом контроле, должны быть описаны.



Описание тегов @class, @public, @implements необязательно, т.к. его можно взять из кода.

Шаг 8. Добавьте два типа демо-примеров

- Первый тип демонстрирует возможности контрола и имеет максимально простой исходный код. Предназначен для быстрого понимания разработчиками о том, что может сделать контрол. Создается один демо-пример для базовых возможностей контрола ([пример](#)) и по одному примеру на каждую опцию ([пример](#)).
- Второй тип ([пример](#)) необходим для проверки отделом проектирования на соответствие стандартам. Он может представлять из себя все прочие демо-примеры, собранные на одной странице.

При написании демо-примера старайтесь минимизировать количество контейнеров стилей и кода и максимально использовать готовые классы из [файла](#). Очень желательно, чтобы в демо-примере были только рассматриваемые контролы.

Добавление модулей с демо-примерами для контрола на wi.sbis.ru описано в [инструкции](#).

Как организовать исходный код контролов

Одиночный контрол

В случае, когда нам необходимо ввести в наше приложение новый одиночный компонент, то необходимо соблюдать следующие правила структуры файлов:

1. Необходимо расположить файл с логикой компонента в нужной папке модуля и именовать его логически правильным названием в **CamelCase**.

Правильно:



Неправильно:

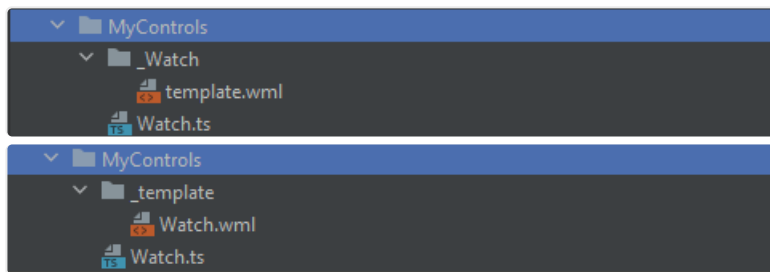


2. Шаблон компонента должен лежать в папке **_НазваниеКомпонента** (это необходимо для паковки), и иметь такое же название как и компонент.

Правильно:

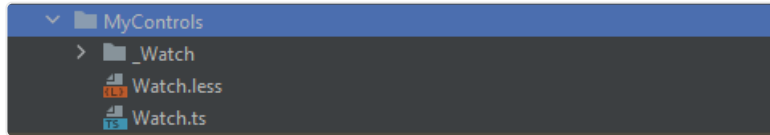


Неправильно:

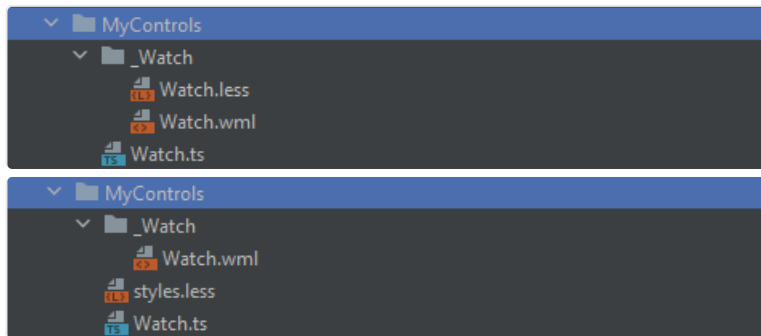


3. Стилизация компонента должна быть расположена на одном уровне с компонентом и иметь такое же имя.

Правильно:



Неправильно:



См. [пример одиночного компонента](#).

Библиотечный контрол

При разработке библиотек нужно соблюдать следующие правила:

1. Файл библиотеки должен располагаться, как и файл контрола, в правильном месте в модуле и именоваться в **lowerCamelCase**.

Правильно:

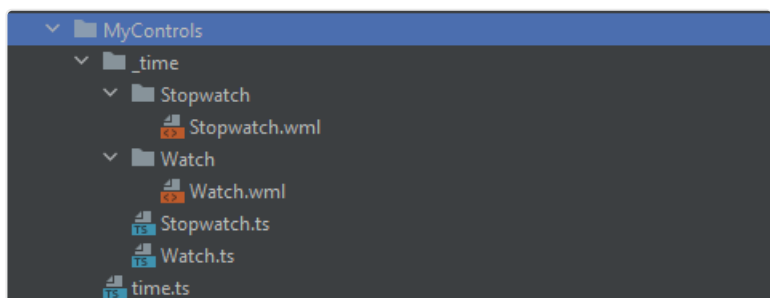


Неправильно:



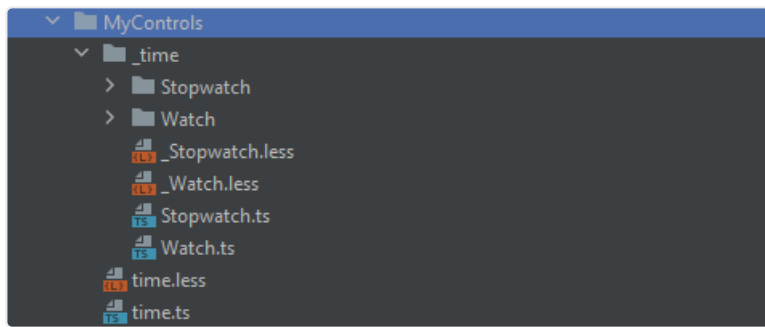
2. Все модули, входящие в библиотеку должны располагаться внутри нее в папке с названием **_названиеБиблиотеки** (достаточно, чтобы имя папки верхнего уровня начиналось с символа нижнего подчеркивания "_", а во вложенных директориях (папки модулей и др) теперь необязательно использовать "_").

Правильно:



3. Стилизация библиотеки должна размещаться на уровне библиотеки и подключать все внутренние less файлы. Файлы стилизации компонентов должны быть именованы **_НазваниеКомпонента** (это необходимо, чтобы эти файлы не собирались в продакшн).

Правильно:



4. Не нужно размещать компоненты, видимые при загрузке страницы, а также компоненты, не видимые при загрузке страницы в одной и той же библиотеке, так как скорее всего вы нарушаете принцип единственной ответственности. Для таких случаев нужно разбивать библиотеки на более меньшие библиотеки.

См. [пример библиотеки](#).

Как организовать исходный код демо-примеров

- В модуле Controls-demo создаем папку, соответствующую названию библиотеки (если в библиотеке больше одного контрола, создаем папку с названием контрола).
- Для каждого демо-примера создаем папку с названием (например, это может быть имя его опции).
- Все файлы должны лежать в пределах одной папки.
- Не может быть двух демо-примеров в одной папке (это условие для корректного отображения исходников на сайте с документацией).

Как добавить демо-примеры на wi.sbis.ru

Важно!

В настоящее время поддерживаются только платформенные модули, так как при добавлении модуля с прикладными зависимостями, может потребоваться подключение огромного количества зависимостей.

См. [пример статьи с демо-примером](#) и [демо-пример](#).

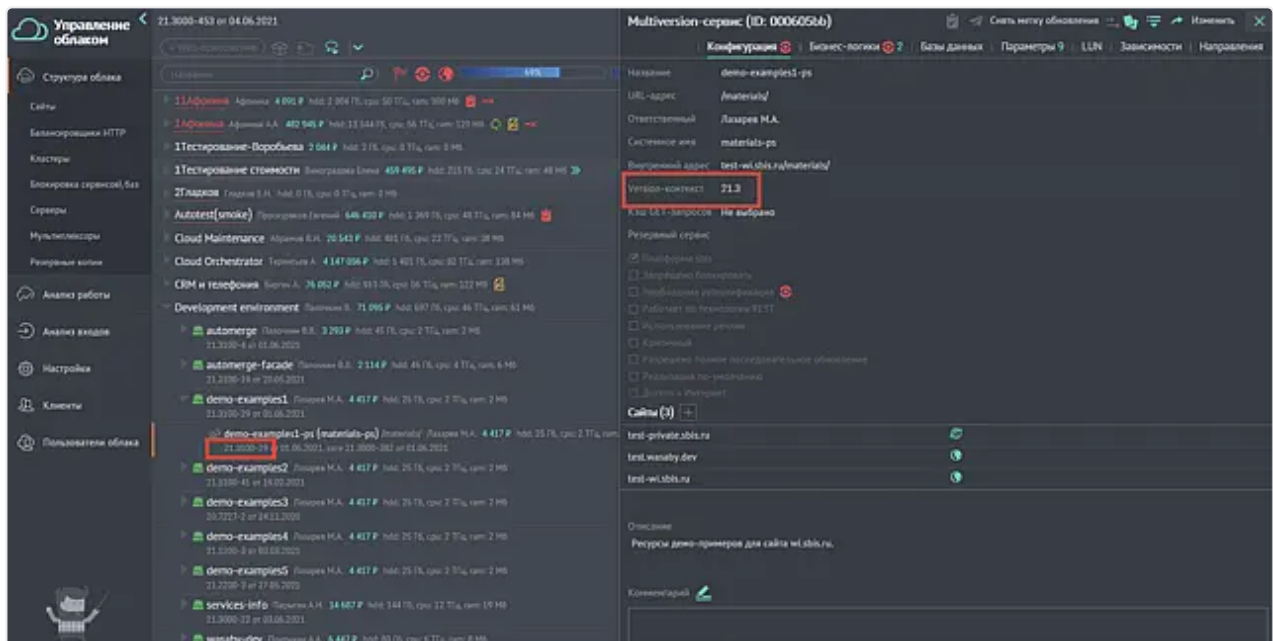
1. Добавьте необходимые модули в файл [platform-demo.s3srv](#).
2. Репозиторий, в котором содержатся модули, должен быть подключен к сборкам **wi.sbis-doc** и **demo-examples**.

Чтобы проверить так ли это, найдите на ci.sbis.ru в сборочных директориях `wi.sbis-doc_{нужная вам версия}` и `demo-examples_{нужная вам версия}` файл [meta.info](#). В этом файле содержатся все подключенные в сборку репозитории (см. пример в `wi.sbis-doc_21.3100` и `demo-examples_21.3100`). Если вашего репозитория там нет, оформите задачу [Добавление репозитория исходного кода](#) со ссылкой на git.sbis.ru, чтобы добавить его в сборки сервисов **wi.sbis-doc** и **demo-examples**.

3. При оформлении [Merge request](#) в поле "Что обновлять" укажите **wi.sbis-doc** и **demo-examples{номер}**.

Чтобы определить, какой из `demo-examples` (`demo-examples1`, `demo-examples2` и т.д.) указывать на обновление, нужно в [облаке](#) найти `demo-examples` с нужной версией и соответствующим ей Version-контекстом (например, для версии 21.3100 Version-контекст - 21.3).

Если сервиса `demo-examples` нужной вам версии еще нет, то укажите только `wi.sbis-doc`.



Как добавить демо-примеры на веб-страницу API класса компонента

См. [Создание документации API TypeScript](#).

Чтобы демо-пример отображался на странице API класса под заголовком "Примеры", нужно воспользоваться тегом `@demo` ([пример](#)). Тег `@demo` следует использовать в доклете, которым описывают класс компонента (см. тег `@class`). В одном доклете допускается использование нескольких тегов `@demo`.

```
1 // Формат тега.
2 @demo <name> [<description>]
3 // Формат тега, если демка находится в прикладном репозитории.
4 @demo <name> [vdom] [<description>]
```

- Параметром `<name>` задают имя компонента, отображаемого демо-примером.
- Опционально, параметром `<description>` задают описание, отображаемое перед демо-примером.

```
1 /**
2  * @class SBIS3.CONTROLS/Button
3  * @demo Examples/Buttons/SimpleButtons Примеры контурных кнопок.
4  * @demo Examples/Buttons/IconButtons Примеры кнопок с иконками.
5  */
```

По умолчанию не всякий компонент можно показать в качестве демо-примера. Интерфейсный модуль, в котором расположен ваш компонент, должен быть включен в состав сервиса `wi.sbis-doc-ps`, чтобы автодокументация появилась в разделе [«Документации по API»](#). Со списком добавленных модулей с демо-примерами и их зависимостями можно ознакомиться в [файле](#).

[Для добавления модуля оформите задачу в разработке на Шубина А. В. с текстом «Прошу добавить в «Документацию по API» модуль ...» и дайте ссылку на \[git.sbis.ru\]\(https://github.com/sbis3/wi.sbis.ru\).](#)

См. пункт 7 в инструкции ["Разработать компонент"](#).

Как сделать демо-примеры на онлайн с доступом к БЛ

1. В репозитории, в котором лежат контролы на демо-примерах, создайте папку. Имя папки задается путём приставки к названию модуля постфикса `demo`: `{Название_модуля}-demo` ([см. пример](#)).
2. Внутри нее создайте отдельный интерфейсный модуль `s3mod` ([см. пример](#)).
3. Добавьте в него контрол с названием `Index.ts` ([см. пример](#)), это и будет шаблон демо-страницы. Она будет доступна по URL-адресу:

online.sbis.ru/<Название_модуля-demo>

4. Чтобы демо-пример попал в сборку online.sbis.ru, добавьте свой модуль в репозиторий online-inside в файл inside-ps.s3srv. Этот файл содержит интерфейсные модули, входящие в сборку дистрибутива inside (см. [пример](#)).
5. На шаблоне созданной страницы (п. 3) можете располагать контролы с необходимыми настройками для демо-примера. Они будут иметь доступ к БЛ на online.

Как сделать демо-примеры на wasaby-cli

См. [документацию](#).

1. Создайте файл **package.json** (если он отсутствует) в репозитории, в котором вы ведете разработку, с помощью команды:

```
npm init
```

2. Установите утилиту для разворота приложения на wasaby:

```
npm install git+https://git.sbis.ru/saby/wasaby-cli.git#rc-[версия, в  
который вы ведёте разработку] --save-dev
```

3. Выполните шаги 1-3 из инструкции ["Как сделать демо-примеры на онлайн с доступом к БЛ"](#).

4. Запустите локальный демо-стенд по инструкции ["Как запустить демо-примеры"](#).

Если вам необходимо в **локальных демо-примерах обращаться к БЛ** (подменять ее ответы), wasaby-cli предоставляет возможность создавать свои обработчики роутов:

1. Для того чтобы обрабатывать стандартные запросы БЛ sbis, вам необходимо задать в своем **package.json** настройку для wasaby-cli ([см. пример](#)):

```
1 {  
2   "wasaby-cli": {  
3     "expressRoute": {  
4       "/service": "service.js"  
5     }  
6   }  
7 }
```

2. Ваш js модуль должен экспортировать **express.Router** ([см. пример](#)):

```
1 const express = require('express');  
2 const router = express.Router();  
3  
4 router.get('/', (req, res) => {  
5   const {  
6     // Название метода вашей бизнес-логики  
7     method,  
8     // Параметры переданные в метод  
9     params  
10    } = req.body;  
11   res.send({  
12     id: 1,  
13     jsonrpc: '2.0',  
14     protocol: 6,  
15     // Результат, который должен получить компонент от бизнес  
    логики. Он формируется в зависимости от метода и от параметров, если  
    это необходимо  
16     result: jsonResult  
17   });  
18 });
```

19 | `module.exports = router;`

3. При запуске локального демо-стенда запросы БЛ из ваших компонентов будут обрабатываться этим модулем.

Как запустить демо-примеры

1. Клонировать репозиторий с контролами, например, в папку "wasaby-control-example" (все команды в следующих пунктах нужно будет выполнять в этой

```
git@git.sbis.ru:saby/wasaby-control-example.git /path/to/wasaby-control-example
```

2. Установите [Node.js](#) и [npm](#).

3. Установите зависимости:

```
npm install
```

4. Загрузите платформенные модули:

```
npx wasaby-cli loadProject
```

5. Соберите проект:

```
npx wasaby-cli buildProject
```

6. Запустите локальный стенд с демо-примерами. Предварительно сгенерируется разводящая страница:

```
npx wasaby-cli startServer
```

После выполнения команды можно открыть в браузере localhost:1024 с разводящей страницей.

Существует команда, которая может за раз выполнить шаги, описанные выше:

```
npx wasaby-cli initDevEnv
```

Также эта команда запускает Watcher.

Отдельные демо-примеры доступны по адресу <http://localhost:1024/DemoStand/app/><имя модуля>.